

P0811R3: Well-behaved interpolation for numbers and pointers

Audience: LWG; WG14

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 22, 2019

History

Changes in r3:

- Made integer midpoint a template.
- Fixed example implementation of it and discussion.
- Updated to modern library description conventions.
- Added missing `constexpr` in wording.
- Added feature test macro.

Changes in r2:

- Renamed `linear` to `lerp` per LEWG.
- Added `constexpr` and `noexcept` where appropriate.
- Expressed sign computations without multiplication or subtraction for robustness.
- Prohibited spurious NaN results.

Changes in r1:

- Renamed `mid` to `midpoint` and `lint` to `linear` after reflector discussion.
- Extended monotonicity to certain cases with infinite t .
- Added wording.

Introduction

The simple problem of computing a value between two other values is surprisingly subtle in general. This paper proposes library functions to compute the midpoint between two integer, floating-point, or pointer values, as well as a more general routine that interpolates (or extrapolates) between two floating-point values.

These utilities are a pure library extension. With the exception of the pointer versions, they would be valuable and straightforward to implement in C (perhaps via type-generic macros); it would be appropriate to consult with WG14 about including them in the C standard library.

Midpoint

Integers

Computing the (integer) midpoint of two integer values via

$(a+b)/2$

can cause overflow for signed or unsigned integers as well as for floating-point values. [Java's binary search implementation](#) had this integer overflow bug for nearly a decade, and Mozilla had [the same issue](#) in its JavaScript implementation.

The standard alternative

$a+(b-a)/2$

works for signed integers with the same sign (even if $b < a$), but can overflow if they have different signs. The modular arithmetic of unsigned integers does not produce the value expected for $b < a$ because the division inherent to midpoint is [not native there](#); it instead produces the value halfway between a and the smallest **modular equivalent** to b that is no smaller.

Using the C++20 conversion from unsigned to signed that preserves bit patterns, the library can provide the simple implementation

```
constexpr Integer midpoint(Integer a, Integer b) noexcept {  
    using U = make_unsigned_t<Integer>;  
    return a > b ? a - (U(a) - b) / 2 : a + (U(b) - a) / 2;  
}
```

that works for signed or unsigned Integer. Note that when $b == a + 1$ or $b == a - 1$ (without overflow), the result is a because of truncating division. This can be exploited to round half-integers up or down by supplying $a >= b$ or $a <= b$ respectively. (The simple $(a+b)/2$ always truncates half-integers towards 0, yielding $\min(a, b)$ when they differ by 1.)

Pointers

When an array is partitioned via pointer arithmetic (for binary search or parallelization, for example), the array size must not exceed PTRDIFF_MAX to avoid undefined behavior ([expr.add]/5). The library can also provide a function template

```
template<class T>  
constexpr T* midpoint(T *a, T *b);
```

which is straightforward on common architectures but, it seems, **cannot be implemented** portably and efficiently in C++. As with integers, when the midpoint lies between two pointer values the one closer to a is chosen; for the usual case of $a < b$, this is compatible with the usual half-open ranges by selecting a when $[a, b)$ is $[a, a+1)$.

Floating-point types

Each of the midpoint formulas above can cause overflow for floating-point types; the latter can also produce results that are not correctly rounded (by rounding in the subtraction and the addition). A third choice

$a/2 + b/2$

prevents overflow but is not correctly rounded for subnormal inputs (due to rounding each separately). The library can easily provide overloads of midpoint for floating-point types that is correctly rounded for all inputs by switching between these strategies:

```
Float midpoint(Float a, Float b)  
{return isnormal(a) && isnormal(b) ? a/2 + b/2 : (a+b)/2;}
```

(P0533R2 proposes making `isnormal constexpr`, in which case this implementation could be.)

Naming

The name `mean` is superior for the floating-point case, but for the other types (and the common application to binary search) the name `midpoint` used above is more appropriate. It would be possible to use both names (despite the possible confusion with the use of `mean` in `[rand.dist]`), but as it aims to replace the simple expression $a+(b-a)/2$ regardless of type, a single name seems best.

Linear interpolation

Both [obvious approaches](#) used in published implementations of floating-point linear interpolation have [issues](#):

1. $a+t*(b-a)$ does not in general reproduce b when $t==1$, and can overflow if a and b have the largest exponent and opposite signs.
2. $t*b+(1-t)*a$ is not monotonic in general (unless the product $ab \leq 0$).

Lacking an obvious, efficient means of obtaining a correctly rounded overall result, the goal is instead to guarantee the basic properties of

1. *exactness*: `lerp(a,b,0)==a && lerp(a,b,1)==b`
2. *monotonicity*: `cmp(lerp(a,b,t2),lerp(a,b,t1)) * cmp(t2,t1) * cmp(b,a) >= 0`, where `cmp` is an arithmetic three-way comparison function
3. *determinacy*: result of NaN only for `lerp(a,a,INFINITY)`
4. *boundedness*: `t<0 || t>1 || isfinite(lerp(a,b,t))`
5. *consistency*: `lerp(a,a,t)==a`

given that each argument `isfinite` (`!isnan` is sufficient for for monotonicity). These properties are useful in proofs of correctness of algorithms based on `lerp`. When interpolating, consistency follows from the other properties, but it and monotonicity apply even when extrapolating.

To demonstrate the feasibility of satisfying these properties, a simple implementation that provides all of them is given here:

```
constexpr Float lerp(Float a, Float b, Float t) {  
    // Exact, monotonic, bounded, determinate, and (for a=b=0) consistent:  
    if(a<=0 && b>=0 || a>=0 && b<=0) return t*b + (1-t)*a;  
  
    if(t==1) return b; // exact  
    // Exact at t=0, monotonic except near t=1,  
    // bounded, determinate, and consistent:  
    const Float x = a + t*(b-a);  
    return t>1 == b>a ? max(b,x) : min(b,x); // monotonic near t=1  
}
```

Putting b first in the `min/max` prefers it to another equal value (i.e., $-0.$ vs. $+0.$), which avoids returning $-0.$ for $t==1$ but $+0.$ for other nearby values of t .

Whether it uses this implementation or not, the library can provide a function satisfying these mathematical properties.

Naming

The name `lerp` is very widely used in the graphics community (including animation and gaming), although in some uses t is restricted to $[0,1]$.

Wording

Relative to N4800.

Feature-test macro

Add at the appropriate place in Table 36:

```
__cpp_lib_interpolate          ... <cmath> <numeric>
```

Midpoint

Add to the end of the synopsis in [numeric.ops.overview]:

```
// 24.9.14, least common multiple
template<class M, class N>
constexpr common_type_t<M,N> lcm(M m, N n);

// 24.9.15, midpoint
template<class T>
constexpr T midpoint(T a, T b) noexcept;
template<class T>
constexpr T* midpoint(T* a, T* b);
}
```

Add a new subsection after [numeric.ops.lcm]:

24.9.15 Midpoint [numeric.ops.midpoint]

```
template<class T>
constexpr T midpoint(T a, T b) noexcept;
```

1. *Constraints*: T is an arithmetic type other than `bool`.
2. *Returns*: Half the sum of a and b . If T is an integer type and the sum is odd, the result is rounded towards a .
3. *Remarks*: No overflow occurs. If T is a floating-point type, at most one inexact operation occurs.

```
template<class T>
constexpr T* midpoint(T* a, T* b);
```

1. *Constraints*: T is a complete object type.
2. *Expects*: a and b point to, respectively, elements $x[i]$ and $x[j]$ of the same array object x . [*Note*: An object that is not an array element is considered to belong to a single-element array for this purpose; see [expr.unary.op]. A pointer past the last element of an array x of n elements is considered to be equivalent to a pointer to a hypothetical element $x[n]$ for this purpose; see [basic.compound]. — *end note*]
3. *Returns*: A pointer to $x[i+(j-i)/2]$, where the result of the division is truncated towards zero.

Linear interpolation

Add to the synopsis in [cmath.syn]:

```
long double fmal(long double x, long double y, long double z);
```

```
// 25.8.4, linear interpolation
constexpr float lerp(float a, float b, float t);
constexpr double lerp(double a, double b, double t);
constexpr long double lerp(long double a, long double b, long double t);
```

```
// 25.8.4, classification / comparison functions
```

Add a new subsection after [c.math.hypot3]:

25.8.4 Linear interpolation [c.math.lerp]

```
constexpr float lerp(float a, float b, float t);
constexpr double lerp(double a, double b, double t);
constexpr long double lerp(long double a, long double b, long double t);
```

1. *Returns:* $a+t(b-a)$.
2. *Remarks:* Let r be the value returned. If `isfinite(a) && isfinite(b)`, then:
 1. If `t==0`, then `r==a`.
 2. If `t==1`, then `r==b`.
 3. If `t>=0 && t<=1`, then `isfinite(r)`.
 4. If `isfinite(t) && a==b`, then `r==a`.
 5. If `isfinite(t) || !isnan(t) && b-a!=0`, then `!isnan(r)`.
3. Let $\text{CMP}(x,y)$ be 1 if $x>y$, -1 if $x<y$, and 0 otherwise. For any t_1 and t_2 , the product of $\text{CMP}(\text{lerp}(a,b,t_2), \text{lerp}(a,b,t_1))$, $\text{CMP}(t_2,t_1)$, and $\text{CMP}(b,a)$ is non-negative.

Acknowledgments

Thanks to Howard Hinnant and Dan Sunderland for debugging the suggested implementation of midpoint, and to Dan and Jeff Garland for extra wording review.